



FÍSICA E INTELIGENCIA ARTIFICIAL PARA VIDEOJUEGOS

Apuntes



VÍCTOR MANUEL PEIRÓ MARTÍNEZ
FIIS 3A

Contenido

Tema 1 – Introducción A Unity	3
Ventanas Principales	3
Coordenadas	3
Transformaciones y Atajos	4
Tema 2 – Transformaciones.....	5
Componente Transform	5
Modificar Posición	5
Modificar Rotación.....	5
Modificar Escala.....	5
Movimientos Cinemáticos.....	6
Movimiento	6
Rotación.....	6
Tema 3 – Triggers y UI Básico	7
Componente Rigidbody.....	7
Componente Collider.....	7
Tipos de Interacciones	7
Referencias a Otros Game Objects	7
Creación de UI.....	8
Problemas Con UI	8
Tema 4 – Manejo de Tiempos.....	9
Tema 5 – Físicas	9
Kinematic vs Dynamic	9
Fuerzas.....	9
Materiales Físicos	10
Tema 6 – Detecciones Físicas.....	11
Overlapping	11
Raycasting	11
Tema 7 – Ciclo de Vida de un Game Object	12
Inicialización	12
Activación/Desactivación.....	12
Destrucción.....	12
Tema 8 – Corrutinas	13

Tema 9 – Patrones de Comunicación	14
Tema 10 – Datos Persistentes Entre Escenas.....	15
Tema 11 – Animaciones.....	16
Componente Animator.....	16
Parámetros	16
Tema 12 – Movimientos para Personajes y NPCs.....	17
CharacterController	17
NPCs.....	17
Tema 13 – Inputs Basicos	19
Input Manager	19

Tema 1 – Introducción A Unity

Ventanas Principales

Ventanas Scene/Game:

- **Modo Scene:** Modo de manipulación de los elementos. Espacio de Trabajo
- **Modo Game:** Como se vera tu juego cuando se ejecuta

Ventana Hierarchy: Representación jerárquica de todos los Game Objects

- Creación y Borrado de Game Objects
- Búsqueda Rápida
- Visibilidad y Activación

Ventana de Project/Console:

- **Ventana Project:** Muestra todo los assets del proyecto
 - Organización de assets en carpetas
 - Búsqueda rápida de assets
 - Creación de nuevos assets
 - Importación de assets externos
- **Ventana Console:** Se muestran mensajes de debug, advertencias y errores
 - Visualización de logs, warning y errores
 - Filtrado de mensajes por tipo
 - Capacidad de hacer clic en los errores directamente

Ventana de Inspector: Ventana donde se puede ver y modificar propiedades y componentes de un game object

- Visualización de componentes
- Edición de propiedades
- Adición y Borrado de componentes

Coordenadas

Coordenadas Globales: Coordenadas respecto al eje del mundo. Son absolutas

Coordenadas Locales: Coordenades respecto a un objeto padre. Son relativas

Center: Centro Geométrico del Objeto

Pivot: Punto de pivote del objeto. No tiene porque coincidir con el center

Gizmo: Ayuda a navegar, visualizar y manipular objetos en la escena 3D

- **Vistas Laterales:** Te permite cambiar rápidamente a vistas ortográficas en los ejes X, Y e Z. Útil para alinear objetos
- **Vista en Perspectiva:** Vista predeterminada que proporciona una representación tridimensional del espacio

Transformaciones y Atajos

W: Activa la herramienta de movimiento permitiendo desplazar objetos

E: Activa la herramienta de rotación permitiendo rotar objetos

R: Activa la herramienta de escalado permitiendo escalar objetos

T: Escalar objetos en una única dirección

Ctrl + D: Duplicar un Game Object

F: Enfocar la vista de la cámara en el objeto seleccionado

Ctrl + Shift: Maximiza la ventana activa

V: Permite alinear vértices de distintos objetos

Ctrl + W/E/R: Aplica una transformación discreta

- **W:** Se mueve en incremento de unidad
- **E:** Se rota en incrementos de 15 grados
- **R:** Se escala en incrementos de 0.1

Tema 2 – Transformaciones

Componente Transform

Componente Transform: Define Posición, Rotación y escalado de un Game Object

Unidades: Una unidad de Unity equivale a 1 metro o a un ángulo

Acceso Por Código: `this.gameObject.transform`

- Si omitimos `this.gameObject` se coje el objeto que lleva el script

No podemos modificar una única coordenada tenemos que modificar las 3

.magnitude: Magnitud de un vector

Modificar Posición

Propiedad `transform.position`: Representa la posición de un objeto con un `Vector3`

No se puede modificar cada componente(X/Y/Z) individualmente

Usa coordenadas globales

Para usar coordenadas locales: `transform.localPosition`

```
transform.position = new Vector3(5f, 2f, 3f); // Modificar componentes de forma individual
transform.position = new Vector3(transform.position.x, 10f, transform.position.z);
```

Modificar Rotación

Se puede manejar la rotación de 2 maneras

- **`transform.eulerAngles`:** Representa un `Vector3` que representa la rotación con grados alrededor de x,y,z

```
transform.eulerAngles = new Vector3(0f, 45f, 0f);
```

- **`transform.rotation`:** Representa la rotación alrededor de un cuaternion

```
transform.rotation = Quaternion.Euler(0f, 90f, 0f);
```

Modificar Escala

Propiedad `transform.localScale`: Permite modificar el tamaño de objeto

Cambiar la escala de un objeto afecta a todos sus hijos

Movimientos Cinemáticos

Movimientos Cinemáticos: Técnicas de movimiento sin usar simulaciones físicas

Movimiento

transform.Translate(dirección * velocidad * Time.deltaTime): Función para mover un objeto

- **dirección:** Vector3 que indica la dirección de movimiento
- **velocidad:** Velocidad del movimiento en unidades
- **Time.deltaTime:** Obligatorio para que se mueva por segundo
 - Si no usásemos esto, dependería de los frames el movimiento

Direcciones Unitarias:

- **Vector3.forward:** Vector (0,0,1)
- **Vector3.back:** Vector (0,0,-1)
- **Vector3.up:** Vector (0,1,0)
- **Vector3.down:** Vector (0,-1,0)
- **Vector3.right:** Vector (1,0,0)
- **Vector3.left:** Vector (-1,0,0)

Hay que normalizar los vectores con **.normalized**

Por defecto se mueve en espacio local pero podemos poner otro parámetro con **Space.World** para que tenga orientación global

Rotación

Transform.Rotate(): Rota un gameObject

Podemos establecer una velocidad común a todos los ejes

```
transform.Rotate ( direccionRot * velRot * Time.deltaTime );
```

O a cada eje:

```
transform.Rotate ( new Vector3(velRotX, velRotY, velRotZ) * Time.deltaTime;
```

Tema 3 – Triggers y UI Básico

Componente Rigidbody

El componente Rigidbody le da físicas a un gameObject

- Hace que este pueda generar interacciones con otros como pueden ser choques o atravesar objetos
- Le da gravedad al objeto

Componente Collider

El componente collider define una área de interacción.

Existen bastantes formas de colliders

- Box Collider
- Sphere Collider
- Capsule Collider

Tipos de Interacciones

Trigger: Dos Objetos se atraviesan

- **Condiciones:**
 - Al menos un objeto tiene que ser un rigidbody
 - Ambos objetos deben llevar un collider
 - Al menos uno debe estar marcado como un trigger
- **Fases:**
 - **OnTriggerEnter:** Entrada al trigger. Solo una vez
 - **OnTriggerStay:** Permanencia en el trigger
 - **OnTriggerExit:** Salida al trigger. Solo una vez

Colisión: Dos objetos se chocan/golpean

Referencias a Otros Game Objects

Los métodos de colisión reciben como parámetro el collider del otro game object

Se puede acceder al otro game object con **other.gameObject**

Destroy(): Destruye game objects

Tags: Usadas para diferenciar con que nos colisionamos

other.gameObject.CompareTag(): Método para comparar etiquetas

Creación de UI

Canvas: Lienzo que se renderiza en la cámara de nuestro juego

Todo elemento de UI viene con un **RectTransform**



Problemas Con UI

Diferentes Resoluciones:

- **Solución:** Hacer los elementos proporcionales a la cantidad cambiando el Scale Mode a Scale with Screen Size

Diferentes Relaciones de Aspecto:

- **Solución:** Anclas

Anclas:

- La posición de cada elemento es relativa a otro elemento
- **Anclas Juntas:**
 - El tamaño del elemento es fijo
 - Las anclas atraen en elementos
 - Configuración común
- **Anclas Separadas:**
 - El tamaño se reescala en proporción a la pantalla
 - También permiten distribuir los elementos por porcentajes
- **Presets:**
 - Configuraciones más comunes
 - Anclaje en laterales
 - Extender respecto al padre

Elementos:

- **Texto:** TMP_Text
- **Imágenes:** Image

Tema 4 – Manejo de Tiempos

La forma mas sencilla de contar en nuestro juego a partir de `Time.deltaTime`

Time.deltaTime: Por segundo

Para que un timer sea efectivo, siempre ha de ser actualizado en el `Update`

Podemos usar booleanos para controlar los contadores

Tema 5 – Físicas

Kinematic vs Dynamic

Queremos usar movimientos dinámicos para que los movimientos son mas realistas

Un `Rigidbody` dinámico no debe de usar comportamientos cinématicos

- Tampoco se puede mover un objeto cinemático por fuerzas

Para aplicar un movimiento dinámico debemos aplicar una fuerza a un `Rigidbody`

Fuerzas

Nota: Ejecutar todas las físicas continuas en `FixedUpdate()`. Se reproduce cada 0,02s

rb.AddForce(dirección* fuerza, ForceMode.TipoFuerza): Metodo para añadir fuerzas

- **rb:** `Rigidbody` al que se el aplica la fuerza
- **dirección:** Direccion de la fuerza
- **fuerza:** Fuerza a aplicar en Newtones
- **TipoFuerza:** Tipo de Fuerza a Aplicar

Tipos de Fuerza:

- **ForceMode.Force:** Fuerza constante de arrastre/empuje. Requiere actualización
- **ForceMode.Impulse:** Fuerza instantánea de golpe
- **ForceMode.Acceleration:** Fuerza constante de arrastre/empuje sin tener en cuenta la masa
- **ForceMode.VelocityChange:** Fuerza instantánea de golpe, sin tener en cuenta la masa

rb.AddTorque(dirección * fuerza, modoFuerza): Aplica una fuerza de rotación

- **rb:** `Rigidbody` al que se el aplica la fuerza
- **dirección:** Direccion de la fuerza
- **fuerza:** Fuerza a aplicar en Newtones
- **TipoFuerza:** Tipo de Fuerza a Aplicar

rb.linearVelocity: Velocidad lineal de un objeto.

- Se puede sobrescribir
- Muy útil para movimientos cinemáticos
- No hace falta actualizarlo

rb.isKinematic: Establece si un Rigidbody es cinemático o no

Materiales Físicos

Podemos crear ciertas reglas físicas cuando 2 materiales interactúan.

- Suelos Deslizantes
- Suelos Rebotantes

Se crean materiales físicos para definir estas reglas

Estos materiales se añaden a su collider

Propiedades:

- **Dynamic Friction:** Fricción cuando el objeto ya está en movimiento
- **Static Friction:** Fricción cuando el objeto todavía está estático
- **Bounciness:** Rebota a Aplicar
- **Combine:**
 - **Average:** Se coge la media de ambos materiales físicos
 - **Min/Max:** Se escoge el valor mínimo/máximo de ambos materiales
 - **Multiply:** Se multiplican los valores de las propiedades de ambos materiales

Tema 6 – Detecciones Físicas

Podemos usar estos algoritmos para detección instantánea

Overlapping

Physics.OverlapSphere(posOverlap, radio, capas): Para detectar colisiones en una esfera

- **posOverlap:** Posición donde estará la esfera de localización
- **radio:** Radio de la esfera
- **conjuntoCapas:** Capas a las que afecta la detección
- Devuelve el conjunto de colliders detectados que pertenecen al conjunto de capas

Gizmos.DrawSphere: Para ver el área de detección

Raycasting

Physics.Raycast(posRay, dirRay, out hitInfo, distancia): Para detecciones en un solo punto

- **posRay:** Posición de donde sale el rayo
- **dirRay:** Dirección del rayo
- **hitInfo:** Variable de tipo RaycastHit que almacena la información del impacto
- **distancia:** Distancia a la que llega el rayo

Debug.DrawRay(): Para dibujar el rayo

Tema 7 – Ciclo de Vida de un Game Object

Inicialización

Métodos de inicialización:

- **Awake():**
 - Siempre se ejecuta antes de cualquier Start()
 - Se ejecuta aunque el script este desactivado
- **OnEnable()**
- **Start():**
 - Para inicializaciones que dependan de otros objetos ya preparados

Instantiate(prefab, position, rotation): Para inicializar para un objeto en tiempo de ejecución

- **prefab:** Referencia al prefab a instanciar
- **position:** Posición de la instancia de la copia
- **rotation:** Quaternion que indica la rotación con la que se indica la copia

Activación/Desactivación

Una vez inicializado, un gameObject puede ser activado o desactivado

SetActive(true/false): Activa y desactiva un objeto

Un objeto desactivado desactiva tmb sus componentes por lo que no se renderizan sus scripts

GetComponent<Componente>(): Para obtener un componente de un gameObject

Podemos activar/desactivar solo ciertas partes usando **.enabled**

No todos los componentes pueden ser desactivados.

Los scripts son componentes por lo que se puede activar/desactivar

Destrucción

Destroy(): Destrucción de gameObjects

- Si metemos un float se destruye después de cierto tiempo

Tema 8 – Corrutinas

A la hora de hacer tareas mas complejas en tiempo, es preferible usar corrutinas

Corrutina: Método que se ejecuta por intervalos de tiempo

IEnumerator: Para establecer métodos como corrutinas

yield return new WaitForSeconds(): Para meter esperas en la corrutina

StartCoroutine(): Para inicializar una corrutina

No se debería tener varias corrutinas a la vez

while (true): Bucle infinito

StopCoroutine(): Para parar una corrutina

Tema 9 – Patrones de Comunicación

Referencia Directa: [Serialize Field]

- Permite asignar referencias directamente desde el Inspector de Unity

Referencia Por Búsqueda: Métodos find

- `GameObject.Find()` o `FindObjectOfType()` o `FindWithTag()`
- Operaciones extremadamente costosas
- Generan código frágil
- Solo deben usarse si no hay otra opción de diseño y estrictamente durante la fase de inicialización

Referencia en Interacciones Físicas: Resolución mediante motor de físicas

- Obtención dinámica del game object
- `TryGetComponent(out GameObject)`

Encapsulamiento al Usar Variables de Otras Clases: `public type Var {get; set;}`

- Usamos getters y setters para no romper encapsulación

Patrón Singleton:

```
public class GameManager : MonoBehaviour
{
    2 usages
    public static GameManager Instance { get; private set; }

    Event function
    private void Awake()
    {
        if (Instance == null)
        {
            Instance = this;
            DontDestroyOnLoad(target: this);
        }
        else
        {
            Destroy(gameObject);
        }
    }
}
```

Tipos:

- **Singleton de Juego:** Con `DontDestroyOnLoad()`. Mismo para todo el juego
- **Singleton de Escena:** Solo existe en la escena. Sin `DontDestroyOnLoad()`

Tema 10 – Datos Persistentes Entre Escenas

Al cambiar de escena, Unity destruye todos los game objects

Sin Patrón	Con Singleton Manager
• El Player guarda sus datos en sus scripts • Al cargar la escena, el player viejo es destruido • El player nuevo nace si memoria del anterior • Los datos se pierden	• Guarda datos entre escenas • El nuevo Player pide los datos al GM • Patrón escalable a cualquier dato

En lugar de encapsular todos los datos, se crea una clase PlayerData

PlayerData.cs

```
// No hereda de MonoBehaviour
public class PlayerData
{
    public Vector3 position;
    public Vector3 rotation;
    public int health = 100;
    public int score = 0;
    public int lives = 3;
    // Añade aquí más campos
}
```


Tema 11 – Animaciones

Componente Animator

Animator: Encargado de gestionar las animaciones de un game object

Animator Controller: Archivo que contiene la maquina de estados de animaciones

Animation Clip: Cada animación completa

Parámetros

Trigger: Parámetro tipo pulsación. Se activa una vez y se reinicia automáticamente tras disparar la transición

- Preferible para animaciones que se ejecutan un número determinado de veces
- Se activa con **SetTrigger()**
- No tiene valor persistente, esta o pendiente o consumido

Bool: Mantiene un valor true/false de manera persistente

- Preferible para animaciones de valor indeterminado
- Persistente hasta que se active con **SetBool()**

Has Exit Time: Cuando activada, Unity espera a que se llegue a un porcentaje de la animación antes de transicionar

- La transición no ocurre hasta que el clip no llega a el exit time

Transition Duration: Tiempo de mezcla entre animaciones

Tema 12 – Movimientos para Personajes y NPCs

CharacterController

Hibrido: Combina el control absoluto del transform con la detección de entorno de rigidbody

Sin Fisica Real: No experimenta inercia

Restricciones: No atraviesa paredes ni suelos

Uso: Personajes humanoides

Slope Limit: Máxima inclinación escalable

Step Offset: Altura Maxima de escalón automático

Skin Width: Margen de seguridad anti-penetración

Move(velocity* Time.deltaTime): Metodo de movimiento

- **Velocity:** Vector de velocidad

Solo una llamada por frame

Detección de Colisiones: Metodo onControllerColliderHit

Característica	CharacterController	Rigidbody
Control	Preciso / Instantáneo (FPS)	Simulado / Con Inercia
Gravedad	Cálculo Manual (Código)	Automática (Built-in)
Colisiones	No empuja por defecto	Reacción física real
Escaleras	Sube automáticamente (Step Offset)	Requiere física compleja / Rampas

NPCs

NavMeshSurface: El escenario

- **Bake:** Analiza la geometría estática del nivel
 - Determina que áreas son transitables y cuales son obstáculos

NavMeshAgent: Componente esencial para npcs/enemigos

- Calcula rutas usando A*
- Desplaza al personaje evitando obstáculos
- **Speed:** Velocidad máxima de movimiento lineal
- **Acceleration:** Que tan rápido alcanza la v max
- **Angular Speed:** Velocidad de rotación
- **Stopping Distance:** Distancia a la que se detiene antes de tocar el objetivo

SetDestination(): Calcula la ruta mas corta a una posición

Verificación De Llegada: 2 condiciones

1. Si la ruta ha terminado de calcularse
2. La distancia restante es menor a la de frenado

```
if (!agent.pathPending &&  
    agent.remainingDistance <= agent.stoppingDistance)
```

Tema 13 – Inputs Basicos

Unity ofrece 2 sistemas de gestión de entrada del jugador

Input Manager (Sistema Estándar): Mas simple y directo para aprender conceptos básicos de detección de entradas

Input System (Sistema Nuevo): Mas flexible y potente, permite configurar controles más y adaptables a diferentes dispositivos

Input Manager

Input.GetKeyDown(): Devuelve true en el primer frame donde se toca la tecla

Input.GetKey(): Devuelve true cuando se mantiene presionada

Input.GetKeyUp(): Se activa en el ultimo frame

Input.GetAxis(): Devuelve un float entre -1 y 1 permitiendo transición gradual entre los extremos

Input.GetAxisRaw(): Devuelve valores enteros entre -1, 0 y 1. Movimiento mas inmediato y preciso

Tema 14 – Patron FSM

FSM: Modelo de comportamiento donde un agente solo puede estar en un estado activo a la vez y puede transitar entre estados